

STACK

Stack

Stack ADT



Last-In-First-Out (LIFO)

Introduction

Stack ADT

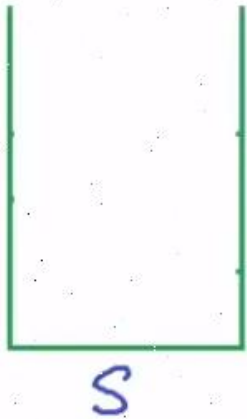
A list with the restriction that insertion and deletion can be performed only from one end, called the top.

Operations

- (1) Push(x)
- (2) Pop()
- (3) Top()
- (4) IsEmpty()

Operations in Stack

Stack ADT (LIFO)



Push(2)
Push(10)
Pop()
Push(7)
Push(5)
Top() $\Rightarrow$ 5
IsEmpty() $\Rightarrow$ false
Pop()
Pop()
Pop()

Operations

- (1) Push(x)
- (2) Pop()
- (3) Top()
- (4) IsEmpty()

Applications

Applications

- Function Calls / Recursion
- undo in an editor
- Balanced Parentheses

{ () }

Stack Implementation

We can implement stacks using:

- a) Arrays
- b) Linked Lists

Stack – Array based implementation

Array implementation

```
int A[10]
```

```
top ← -1 //empty stack
```

```
Push(x)
```

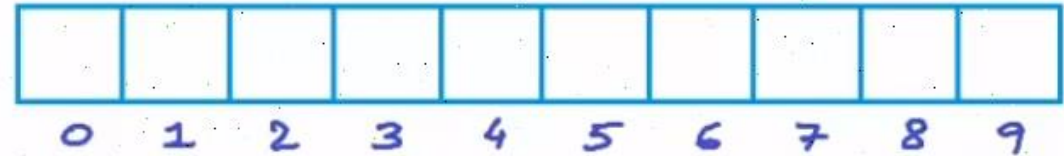
```
{
```

```
    top ← top + 1
```

```
    A[top] ← x
```

```
}
```

top
↓



Stack – Array based implementation

Array implementation

```
int A[10]
```

```
top ← -1 //empty stack
```

```
Push(x)
```

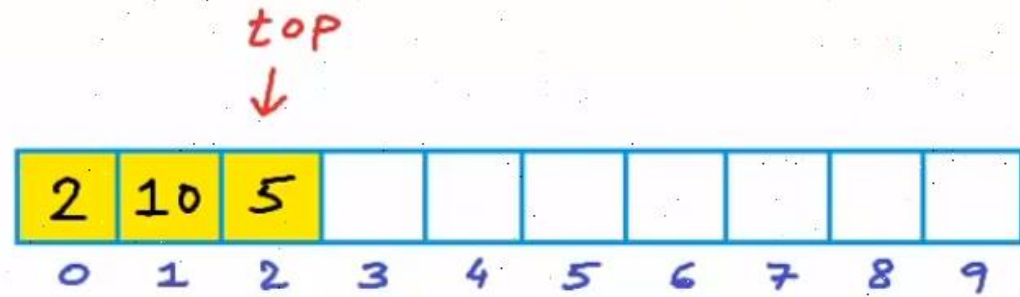
```
{  
  top ← top + 1
```

```
  A[top] ← x
```

```
}
```

```
Pop()
```

```
{  
  top ← top - 1  
}
```



Push(2)

Push(10)

Push(5)

Stack – Array based implementation

Array implementation

```
int A[10]
```

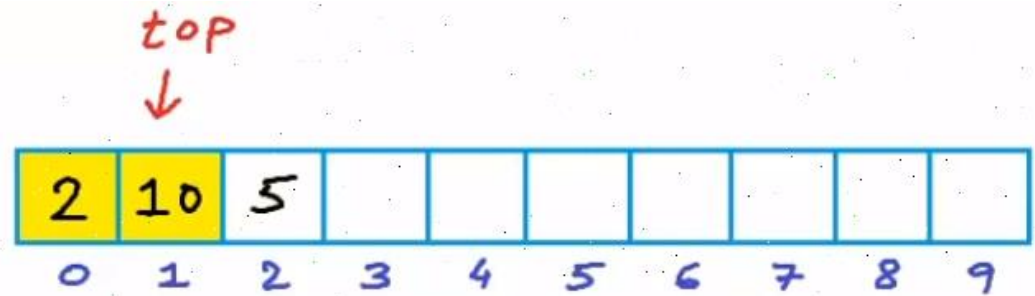
```
top ← -1 //empty stack
```

```
Push(x)
```

```
{  
  top ← top + 1  
  A[top] ← x  
}
```

```
Pop()
```

```
{  
  top ← top - 1  
}
```



Push(2)

Push(10)

Push(5)

Pop()

Push(7)

Stack – Array based implementation

Array implementation

```
int A[10]
```

```
top ← -1 //empty stack
```

```
Push(x)
```

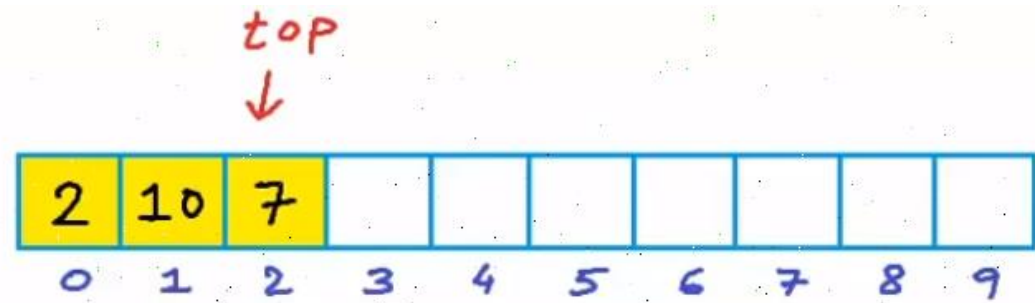
```
{  
  top ← top + 1
```

```
  A[top] ← x
```

```
}
```

```
Pop()
```

```
{  
  top ← top - 1  
}
```



```
Push(2)
```

```
Push(10)
```

```
Push(5)
```

```
Pop()
```

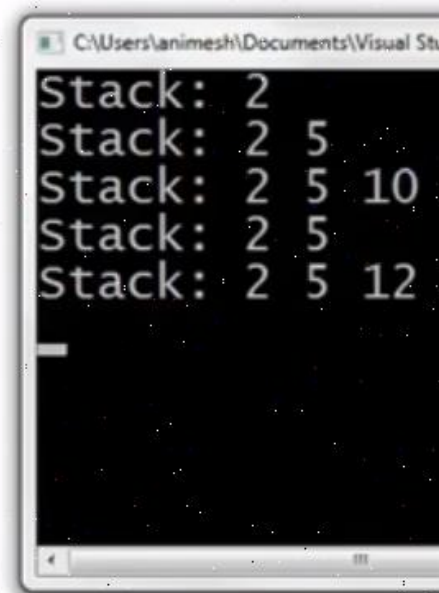
```
Push(7)
```

Stack – Array based implementation

```
// Stack - Array based implementation.
#include<stdio.h>
#define MAX_SIZE 101
int A[MAX_SIZE];
int top = -1;
void Push(int x) {
    if(top == MAX_SIZE -1) {
        printf("Error: stack overflow\n");
        return;
    }
    A[++top] = x;
}
void Pop() {
    if(top == -1) {
        printf("Error: No element to pop\n");
        return;
    }
    top--;
}
```

Stack – Array based implementation

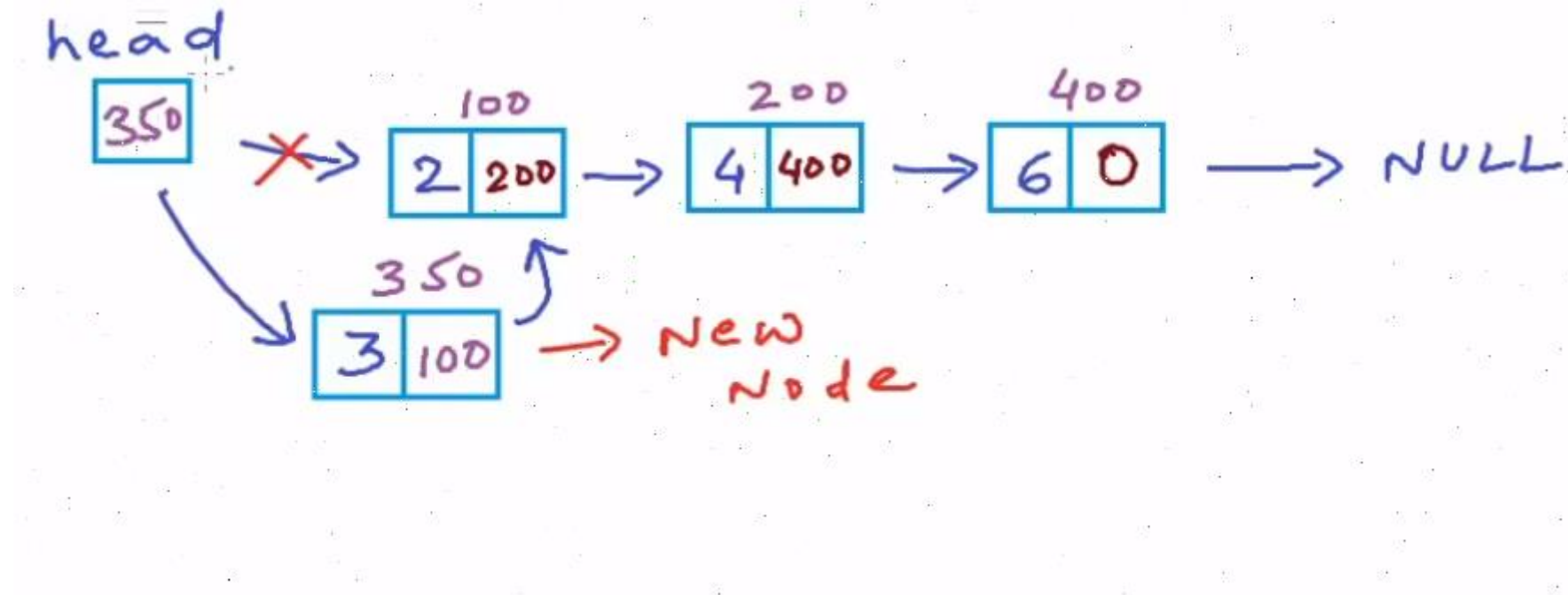
```
void Print() {  
    int i;  
    printf("Stack: ");  
    for(i = 0; i <= top; i++)  
        printf("%d ", A[i]);  
    printf("\n");  
}  
int main() {  
    Push(2); Print();  
    Push(5); Print();  
    Push(10); Print();  
    Pop(); Print();  
    Push(12); Print();  
}
```



A screenshot of a terminal window with a black background and white text. The window title is "C:\Users\animesh\Documents\Visual Stu". The output shows the stack state after each operation: "Stack: 2", "Stack: 2 5", "Stack: 2 5 10", "Stack: 2 5", and "Stack: 2 5 12".

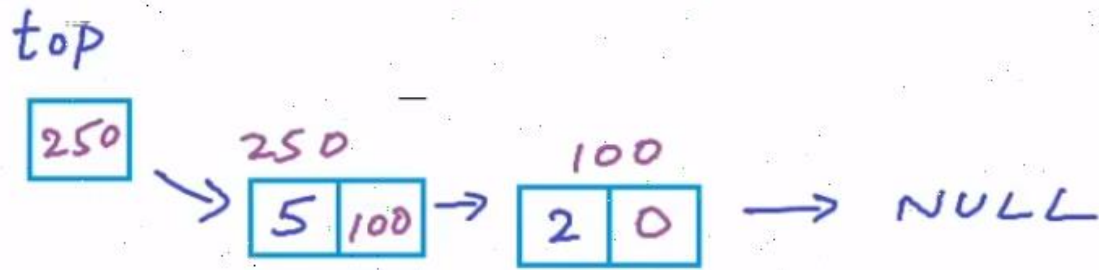
```
Stack: 2  
Stack: 2 5  
Stack: 2 5 10  
Stack: 2 5  
Stack: 2 5 12
```

Stack – Linked List based implementation



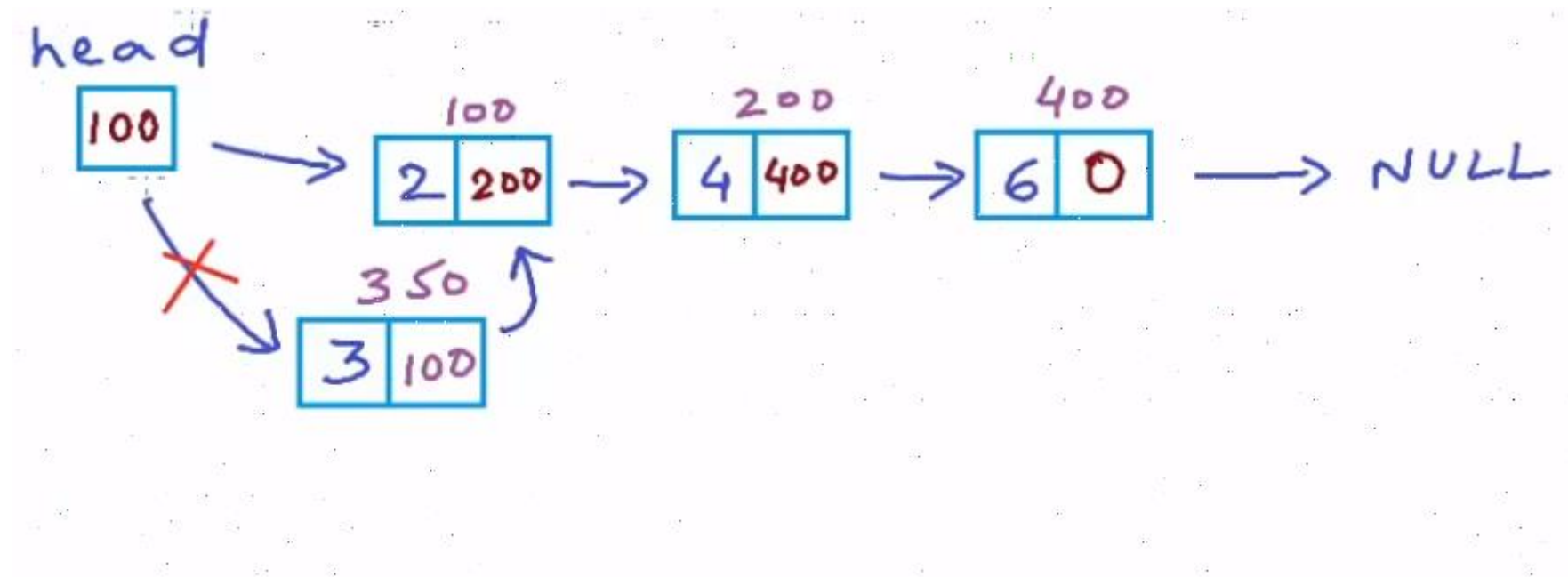
Stack – Linked List based implementation

```
struct Node {  
    int data;  
    struct Node* link;  
};  
struct Node* top = NULL;  
void Push(int x) {  
    struct Node* temp =  
        (struct Node*)malloc(sizeof(struct Node*));  
    temp->data = x;  
    temp->link = top;  
    top = temp;  
}
```



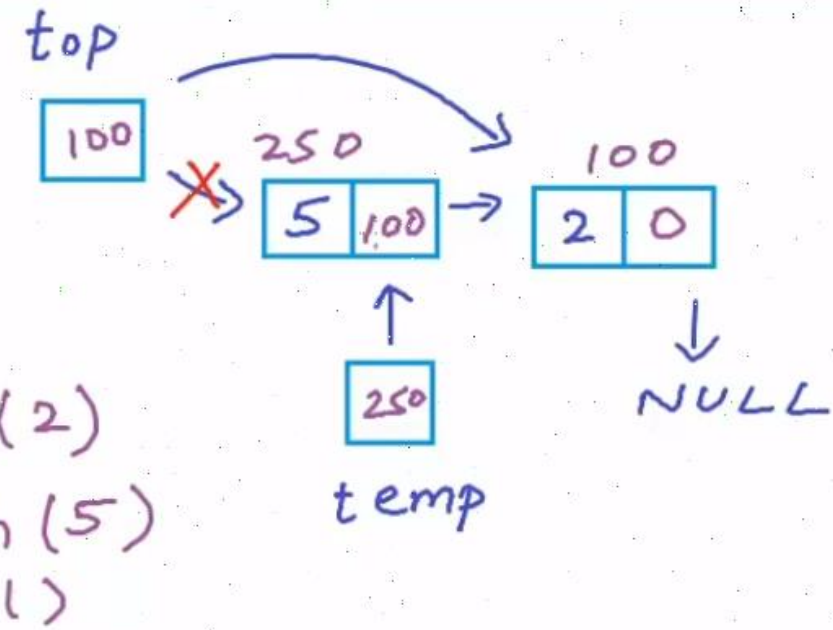
Push(2)
Push(5)

Stack – Linked List based implementation



Stack – Linked List based implementation

```
struct Node* top = NULL;
void Push(int x) {
    struct Node* temp =
        (struct Node*)malloc(sizeof(struct Node*));
    temp->data = x;
    temp->link = top;
    top = temp;
}
void Pop() {
    struct Node *temp;
    if(top == NULL) return;
    temp = top;
    top = top->link;
    free(temp);
}
```



Infix Expression

Infix, Postfix, Prefix

Evaluation of expressions

$2 + 3$

$A - B$

$(P * 2)$

$(2 + 3) * 4$

$(P + Q) * (R + S)$

Infix

$\langle \text{operand} \rangle \langle \text{operator} \rangle \langle \text{operand} \rangle$

Infix evaluation

Infix

$$2 + 3 = 5$$

$$4 + 6 * 2 = 4 + 12 = 16$$

$$2 * 6 / 2 - 3 + 7 = 10$$

Order of operation

- 1) Parentheses $() \{ \} []$
- 2) Exponents (right to left)
- 3) Multiplication and division (left to right)
- 4) Addition and Subtraction (left to right)

Postfix Expression

Postfix
(Reverse polish notation)

$\langle \text{operand} \rangle \langle \text{operand} \rangle \langle \text{operator} \rangle$

Infix

$2 + 3$

$p - q$

$a + b * c$

Prefix

$+ 2 3$

$- p q$

$+ a * b c$

Postfix

$2 3 +$

$p q -$

$a b c * +$

Prefix

Prefix
(Polish notation)

<operator> <operand> <operand>

Infix

$2 + 3$

$p - q$

$a + b * c$

Prefix

$+ 2 3$

$- p q$

$+ a * b c$

Infix to Postfix

$$\begin{array}{c} a + (b * c) \\ \Downarrow \\ a + (bc *) \\ \Downarrow \\ a(bc *) + \\ \Downarrow \\ abc * + \end{array}$$

Infix to Postfix

$$\begin{aligned} \{ \underbrace{(A * B)}_{AB*} + \underbrace{(C * D)}_{cd*} \} - e &\Rightarrow \{ (AB*) + (cd*) \} - e \\ &\Downarrow \\ \{ (AB*) (cd*) + \} - e & \\ &\Downarrow \\ \{ (AB*) (cd*) + \} e - & \\ &\Downarrow \\ AB*cd*+e- & \end{aligned}$$

Infix to Postfix

Infix: $A + (B * C)$

↓
 $A + (B * C)$

↓
 $A (B * C) +$

↓
Postfix: $A B C * +$

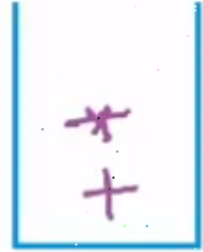
Order of operation

- 1) Parentheses $() \{ \} []$
- 2) Exponents (right to left)
- 3) Multiplication and division (left to right)
- 4) Addition and Subtraction (left to right)

Infix to Postfix

Infix: $A + (B * C) - D * E$

↑



Stack

Postfix: $A B C$

Infix to Postfix

Infix: $A + B * C - D * E$



Stack

Postfix: $ABC*+DE$

Infix to Prefix

$$\begin{array}{c} a + (b * c) \\ \Downarrow \\ a + (*bc) \\ \Downarrow \\ +a(*bc) \\ \Downarrow \\ +a*bc \end{array}$$

Infix to Prefix

$$\{(\underbrace{2 * 3}_{*23}) + (\underbrace{5 * 4}_{*54})\} - 9$$

$$\Rightarrow \{(\underbrace{*23}_{\downarrow}) + (\underbrace{*54}_{\downarrow})\} - 9$$

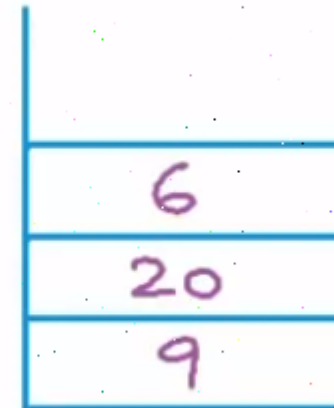
$$\{+ (*23) (*54)\} - 9$$

$$- \{+ (*23) (*54)\} 9$$

$$- + * 2 3 * 5 4 9$$

Infix to Prefix

- + * 2 3 + 5 4 9
↑↑



stack

Evaluating Postfix

$AB * cd * + e -$

$A = 2, B = 3, C = 5$
 $d = 4, e = 9$

$\Rightarrow$ $\boxed{2 \ 3 \ *} \ 5 \ 4 \ * \ + \ 9 \ -$

$\Downarrow$

$6 \ \boxed{5 \ 4 \ *} \ + \ 9 \ -$

$\Downarrow$

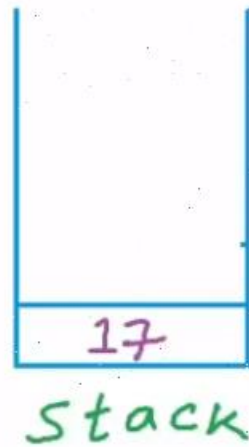
$\boxed{6 \ 20 \ +} \ 9 \ -$

$\Downarrow$

$\boxed{26 \ 9 \ -} \Rightarrow 17$

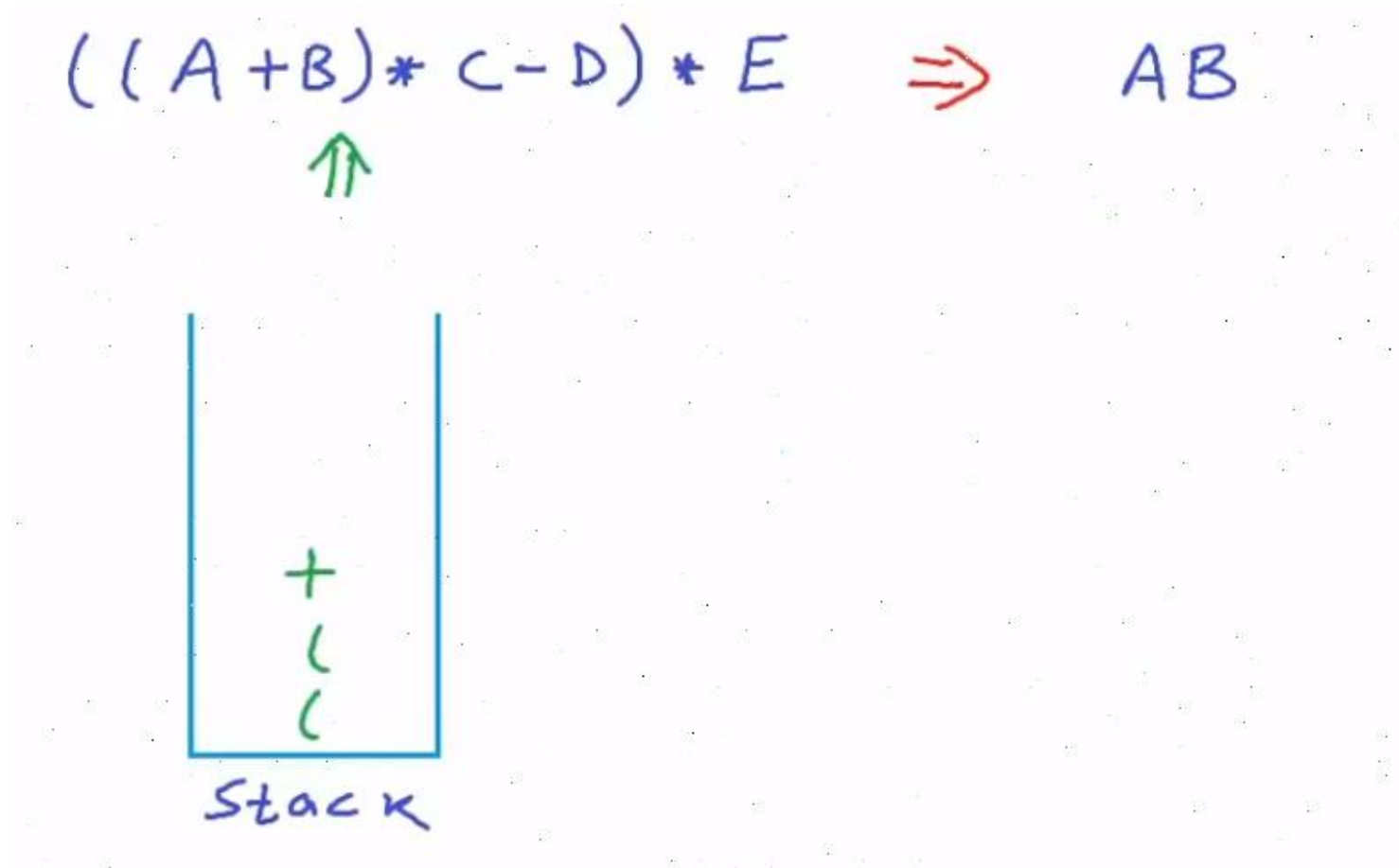
Evaluating Postfix

2 3 * 5 4 * + 9 - $\uparrow\uparrow$



```
EvaluatePostfix(exp)
{
  Create a stack S
  for i ← 0 to length(exp)-1
  {
    if (exp[i] is operand)
      Push(exp[i])
    else if (exp[i] is operator)
    {
      op2 ← Pop()
      op1 ← Pop()
      res ← Perform(exp[i], op1, op2)
      Push(res)
    }
  }
  Return top of Stack
}
```

Infix to Postfix



Infix to Postfix

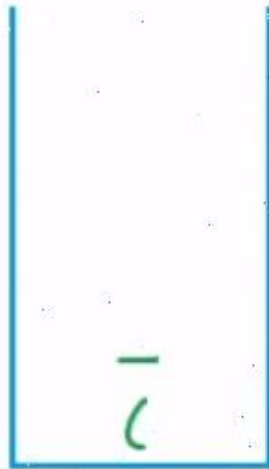
$((A+B)*C-D)*E \Rightarrow AB+C$



Stack

Infix to Postfix

$((A+B)*C-D)*E \Rightarrow AB+C*D$



Stack

Infix to Postfix

$((A+B)*C-D)*E \Rightarrow AB+C*D-E*$

↑

